

Unit - III

Classification

1 Binary Classification: Linear Classification Model

1.1 Binary Classification

Binary classification is a supervised learning task in which each input sample is assigned to one of two possible classes. The class labels are usually represented as

$$y \in \{-1, +1\}$$

or sometimes

$$y \in \{0, 1\}.$$

In binary classification, the learning algorithm uses training data to learn a rule that predicts the class label of new samples.

Each training example is written as

$$(\mathbf{x}_i, y_i)$$

where

$$\mathbf{x}_i \in \mathbb{R}^n$$

is the feature vector and

$$y_i \in \{-1, +1\}$$

is the class label. The index i denotes the training example, with $i = 1, 2, \dots, m$.

Examples of Binary Classification

Binary classification appears in many practical applications:

- **Email filtering:** classifying messages as spam or not spam
- **Medical diagnosis:** determining whether a disease is present or absent
- **Credit approval:** deciding whether to approve or reject a loan
- **Quality control:** identifying defective and non-defective products
- **Exam prediction:** predicting whether a student will pass or fail

1.2 Linear Classification Model

A linear classification model separates two classes using a linear decision boundary. The prediction is based on a linear function of the input features.

The decision function is

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

where

- $\mathbf{x} \in \mathbb{R}^n$ is the feature vector
- $\mathbf{w} \in \mathbb{R}^n$ is the weight vector
- $b \in \mathbb{R}$ is the bias term
- $\mathbf{w}^\top \mathbf{x} = \sum_{j=1}^n w_j x_j$ is the dot product

The value $f(\mathbf{x})$ is called the **decision score**. Its sign determines the predicted class, and its magnitude is proportional to the distance from the decision boundary.

Classification Rule

The predicted class label $\hat{y}$ is determined by the sign of the decision function:

$$\hat{y} = \begin{cases} +1 & \text{if } \mathbf{w}^\top \mathbf{x} + b > 0 \\ -1 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0 \end{cases}$$

If

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

the point lies exactly on the decision boundary. In practice, this case is rare; the classification may be considered ambiguous or assigned arbitrarily to one class.

Decision Boundary

The decision boundary is the set of points satisfying

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

This equation defines a hyperplane that separates the feature space into two regions.

- In two dimensions ($n = 2$), the boundary is a line:

$$w_1 x_1 + w_2 x_2 + b = 0$$

- In three dimensions ($n = 3$), the boundary is a plane:

$$w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0$$

- In higher dimensions ($n \geq 4$), the boundary is called a hyperplane:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

The weight vector $\mathbf{w}$ is perpendicular to the decision boundary and determines its orientation. The bias b determines its position relative to the origin.

Small Numerical Example

Consider a linear classifier with decision function

$$f(\mathbf{x}) = x_1 + x_2 - 3.$$

The decision boundary is

$$x_1 + x_2 - 3 = 0.$$

Classify the point $(2, 1)$:

$$f(2, 1) = 2 + 1 - 3 = 0$$

Since the value equals 0, the point lies exactly on the decision boundary. In practice, this case is rare; the classification may be considered ambiguous or assigned arbitrarily to one class.

Now consider the point $(4, 1)$:

$$f(4, 1) = 4 + 1 - 3 = 2 > 0$$

Therefore the predicted class is $+1$.

Geometric Interpretation

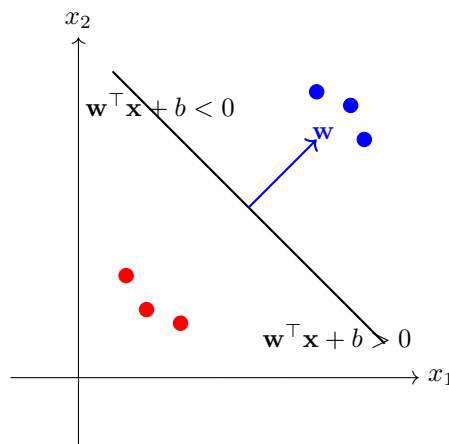


Figure 1: Geometric interpretation of a linear classifier in two dimensions. The vector $\mathbf{w}$ is perpendicular to the decision boundary. Red points belong to class -1 ; blue points belong to class $+1$.

The vector $\mathbf{w}$ is perpendicular to the decision boundary and points toward the region classified as $+1$. The signed distance from a point $\mathbf{x}$ to the boundary is proportional to

$$\frac{f(\mathbf{x})}{\|\mathbf{w}\|}.$$

Illustrative Example

Suppose we classify emails as spam ($+1$) or not spam (-1) using two features:

- x_1 : number of suspicious words
- x_2 : length of the email (in hundreds of words)

Assume the learned parameters are

$$\mathbf{w} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad b = 1.$$

The decision function becomes

$$f(\mathbf{x}) = 2x_1 - x_2 + 1.$$

For an email with

$$x_1 = 3, \quad x_2 = 2$$

$$f(\mathbf{x}) = 2(3) - 2 + 1 = 5$$

Since $f(\mathbf{x}) > 0$, the predicted class is

$$\hat{y} = +1 \quad (\text{spam})$$

For another email with

$$x_1 = 0, \quad x_2 = 5$$

$$f(\mathbf{x}) = 2(0) - 5 + 1 = -4$$

Since $f(\mathbf{x}) < 0$, the predicted class is

$$\hat{y} = -1 \quad (\text{not spam})$$

Practical Considerations

- **Feature scaling:** Linear models are sensitive to the scale of input features. Features with larger magnitudes can dominate the decision function. It is common practice to standardize features to have zero mean and unit variance before training.
- **Feature engineering:** Linear models cannot capture nonlinear relationships unless appropriate feature transformations (e.g., polynomial features, interactions) are provided as inputs.
- **Class imbalance:** When one class significantly outnumbers the other, the decision boundary may be biased. Techniques such as class weighting or resampling can help address this issue.

Advantages of Linear Classification

- **Simplicity:** Easy to understand and implement.
- **Interpretability:** The weight vector directly indicates the importance of each feature.
- **Efficiency:** Fast to train and evaluate, even on large datasets.
- **Well-studied:** Solid mathematical foundation with convex optimization guarantees.

Limitations

- **Linearity assumption:** Cannot capture complex nonlinear patterns without feature engineering.
- **Feature dependence:** Performance depends strongly on the quality and relevance of input features.
- **Sensitivity:** May be sensitive to outliers in some formulations.

Relation to Other Algorithms

Several important machine learning algorithms are based on the linear classification model:

- **Linear Support Vector Machines (SVM):** Find the hyperplane that maximizes the margin between classes.
- **Logistic Regression:** Models class probabilities using the logistic function.
- **Perceptron:** Online learning algorithm that updates weights upon misclassification.
- **Linear Discriminant Analysis (LDA):** Assumes Gaussian class distributions and finds the optimal separating hyperplane.

2 Multi-class Classification

Introduction

Multi-class classification is a supervised learning problem in which each input sample belongs to one of more than two classes. Unlike binary classification, where there are only two classes, multi-class classification involves three or more classes.

Let the class labels be

$$y \in \{1, 2, 3, \dots, K\}$$

where $K \geq 3$ is the number of classes.

Each training example is written as

$$(\mathbf{x}_i, y_i)$$

where $\mathbf{x}_i \in \mathbb{R}^n$ is the feature vector and y_i is the class label.

Model for Multi-class Classification

A linear model for multi-class classification assigns a score to each class. For class k , the score is

$$f_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + b_k$$

where $\mathbf{w}_k$ and b_k are the parameters for class k .

The predicted class is the one with the highest score:

$$\hat{y} = \arg \max_k f_k(\mathbf{x})$$

Confusion Matrix for Multi-class Classification

A confusion matrix summarizes the performance of a classifier. For K classes, it is a $K \times K$ matrix.

	Pred 1	Pred 2	Pred 3
Actual 1	40	5	3
Actual 2	4	35	6
Actual 3	2	7	38

Each row represents actual class and each column represents predicted class.

Per-class Precision and Recall

For each class k , we treat it as a binary problem.

Precision for class k :

$$\text{Precision}_k = \frac{\text{True Positives}_k}{\text{Predicted Positives}_k}$$

Recall for class k :

$$\text{Recall}_k = \frac{\text{True Positives}_k}{\text{Actual Positives}_k}$$

Worked Example

Consider the confusion matrix:

	Pred A	Pred B	Pred C
Actual A	50	5	5
Actual B	10	40	10
Actual C	5	10	45

For class A:

$$\text{Precision}_A = \frac{50}{50 + 10 + 5} = \frac{50}{65} \approx 0.769$$

$$\text{Recall}_A = \frac{50}{50 + 5 + 5} = \frac{50}{60} \approx 0.833$$

For class B:

$$\text{Precision}_B = \frac{40}{5 + 40 + 10} = \frac{40}{55} \approx 0.727$$

$$\text{Recall}_B = \frac{40}{10 + 40 + 10} = \frac{40}{60} \approx 0.667$$

For class C:

$$\text{Precision}_C = \frac{45}{5 + 10 + 45} = \frac{45}{60} = 0.75$$

$$\text{Recall}_C = \frac{45}{5 + 10 + 45} = \frac{45}{60} = 0.75$$

Weighted Average Precision and Recall

Weighted averages consider the number of samples in each class.

Let N_k be the number of samples in class k .

$$\text{Weighted Precision} = \frac{\sum_{k=1}^K N_k \cdot \text{Precision}_k}{\sum_{k=1}^K N_k}$$

$$\text{Weighted Recall} = \frac{\sum_{k=1}^K N_k \cdot \text{Recall}_k}{\sum_{k=1}^K N_k}$$

Compute class counts:

$$N_A = 60, \quad N_B = 60, \quad N_C = 60$$

Weighted Precision:

$$\begin{aligned} &= \frac{60(0.769) + 60(0.727) + 60(0.75)}{180} \\ &= \frac{46.14 + 43.62 + 45}{180} = \frac{134.76}{180} \approx 0.749 \end{aligned}$$

Weighted Recall:

$$\begin{aligned} &= \frac{60(0.833) + 60(0.667) + 60(0.75)}{180} \\ &= \frac{49.98 + 40.02 + 45}{180} = \frac{135}{180} = 0.75 \end{aligned}$$

Handling More Than Two Classes

Multi-class problems can be handled in two ways:

- Use models that directly support multiple classes (e.g., softmax regression)
- Convert the problem into multiple binary classification problems

2.1 One-vs-Rest (OvR)

In One-vs-Rest, we train one classifier per class.

For class k :

- Positive class: class k
- Negative class: all other classes

If there are K classes, we train K classifiers.

Prediction is made by selecting the class with the highest score.

2.2 One-vs-One (OvO)

In One-vs-One, we train a classifier for every pair of classes.

For K classes, the number of classifiers is

$$\frac{K(K-1)}{2}$$

Each classifier distinguishes between two classes only.

Final prediction is made by voting among all classifiers.

Comparison of OvR and OvO

Method	Number of Models	Key Idea
OvR	K	One class vs rest
OvO	$\frac{K(K-1)}{2}$	Pairwise classification

3 Linear Models

Introduction

A **linear model** is a model that makes predictions using a linear relationship between input variables and the output. The model assumes that the output can be expressed as a weighted sum of the input features.

Let the input feature vector be

$$\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n.$$

A general linear model is written as

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

where $x_1, x_2, \dots, x_n$ are input features, $w_1, w_2, \dots, w_n \in \mathbb{R}$ are model weights, $b \in \mathbb{R}$ is the bias term, and $y \in \mathbb{R}$ is the predicted output.

The weights determine the importance of each feature. A larger absolute value of a weight indicates that the corresponding feature has a stronger influence on the prediction.

Meaning of $\mathbb{R}^n$

The symbol $\mathbb{R}$ denotes the set of all real numbers. The notation $\mathbb{R}^n$ represents the set of all vectors that contain n real numbers.

Thus, a vector in $\mathbb{R}^n$ has the form

$$(x_1, x_2, \dots, x_n)$$

where each $x_i \in \mathbb{R}$.

In simple terms, $\mathbb{R}^n$ is called the **n -dimensional real space**.

Examples

$$\mathbb{R}^2 = \{(x_1, x_2) \mid x_1, x_2 \in \mathbb{R}\}$$

represents the two-dimensional plane.

$$\mathbb{R}^3 = \{(x_1, x_2, x_3) \mid x_1, x_2, x_3 \in \mathbb{R}\}$$

represents the three-dimensional space.

Example in $\mathbb{R}^3$

A vector

$$(2, -1, 3.5) \in \mathbb{R}^3$$

contains three real numbers. It represents a point in three-dimensional space.

In machine learning, writing

$$\mathbf{x} \in \mathbb{R}^n$$

means that the feature vector $\mathbf{x}$ contains n real-valued features.

Vector Representation

Linear models are often written using vector notation because it provides a compact mathematical representation. Let

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \in \mathbb{R}^n.$$

The linear model can then be written as

$$y = \mathbf{w}^\top \mathbf{x} + b$$

where $\mathbf{w}^\top$ denotes the transpose of the weight vector. The term $\mathbf{w}^\top \mathbf{x}$ represents the dot product

$$\mathbf{w}^\top \mathbf{x} = \sum_{i=1}^n w_i x_i.$$

Linear Decision Boundary

In classification problems, linear models create a **linear decision boundary**. This boundary separates the feature space into regions that correspond to different class labels.

In two dimensions the decision boundary is a straight line

$$w_1 x_1 + w_2 x_2 + b = 0.$$

In three dimensions the boundary becomes a plane

$$w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0.$$

In higher dimensions the boundary is called a **hyperplane**

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

This hyperplane divides the feature space into two regions.

Example

Suppose a model predicts whether a student passes an exam based on two features: hours studied and number of practice tests completed.

Let

$$x_1 = \text{hours studied}, \quad x_2 = \text{number of practice tests}.$$

Assume the learned model is

$$y = 2x_1 + 3x_2 - 5.$$

For a student with

$$x_1 = 3, \quad x_2 = 2$$

the prediction becomes

$$y = 2(3) + 3(2) - 5 = 7.$$

If positive values correspond to pass and negative values correspond to fail, the student is predicted to pass.

Advantages of Linear Models

Linear models have several advantages. They are easy to interpret because each weight represents the contribution of a feature. They are computationally efficient and can be trained quickly even on large datasets. The mathematical formulation is simple and the optimization problems are usually convex, which guarantees a unique optimal solution.

Limitations of Linear Models

Linear models also have limitations. They cannot represent complex nonlinear relationships unless additional feature transformations are introduced. Their performance depends strongly on the quality of the input features. They can also be sensitive to outliers.

Many machine learning algorithms are based on linear models. Examples include Linear Regression, Logistic Regression, and Linear Support Vector Machines.

4 Linear Support Vector Machines (SVM)

Introduction

The Support Vector Machine (SVM) is a supervised learning algorithm used mainly for classification problems. Linear SVM attempts to find a hyperplane that separates two classes.

The main idea of SVM is to choose the separating hyperplane that maximizes the distance between the classes. This distance is called the **margin**.

Binary Classification Formulation

Consider a binary classification dataset with class labels $+1$ and -1 . Each training example is represented as

$$(\mathbf{x}_i, y_i)$$

where $\mathbf{x}_i \in \mathbb{R}^n$ is the feature vector and $y_i \in \{-1, +1\}$ is the class label.

The decision function of the SVM is

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b.$$

The predicted class is determined using

$$\hat{y} = \begin{cases} +1 & \text{if } \mathbf{w}^\top \mathbf{x} + b > 0 \\ -1 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0 \end{cases}$$

The value $f(\mathbf{x})$ is called the **decision score**. Its sign determines the predicted class. Its magnitude is proportional to the distance from the decision boundary.

Separating Hyperplane

The separating hyperplane is defined by

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

Points on one side of the hyperplane belong to class $+1$. Points on the other side belong to class -1 .

Margin

The margin is the distance between the separating hyperplane and the closest data points from each class.

Two parallel hyperplanes define the margin boundaries

$$\mathbf{w}^\top \mathbf{x} + b = 1$$

$$\mathbf{w}^\top \mathbf{x} + b = -1.$$

The distance between these two hyperplanes is

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}.$$

A smaller value of $\|\mathbf{w}\|$ produces a larger margin.

Support Vectors

The data points that lie closest to the margin boundaries are called **support vectors**. These points determine the position of the separating hyperplane. If the support vectors move, the decision boundary also changes.

4.1 Hard-Margin SVM

In some datasets the two classes can be perfectly separated by a linear boundary. This situation is called **linear separability**. In such cases it is possible to draw a hyperplane that correctly classifies all training points.

However, many different separating hyperplanes may exist. Some of these hyperplanes pass very close to the data points, while others leave a larger gap between the classes.

Support Vector Machines choose the hyperplane that leaves the **largest possible margin** between the two classes. This model is called the **hard-margin SVM** because it does not allow any classification errors.

Margin Constraints

Recall that the separating hyperplane is

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

The margin boundaries are defined as

$$\mathbf{w}^\top \mathbf{x} + b = 1$$

and

$$\mathbf{w}^\top \mathbf{x} + b = -1.$$

For correctly classified points the following condition must hold:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$$

This constraint ensures two things:

- each point is classified correctly

- each point lies outside the margin region

Points that satisfy the equality

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$$

lie exactly on the margin boundaries and become the **support vectors**.

Why Optimization is Needed

Many hyperplanes can separate the data. The goal of SVM is to choose the one that maximizes the margin between the classes.

The width of the margin is

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}.$$

Maximizing the margin is therefore equivalent to minimizing the norm of the weight vector. For mathematical convenience the squared norm is minimized instead.

Hard-Margin Optimization Problem

The SVM training problem becomes the following constrained optimization problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, m.$$

The objective function

$$\frac{1}{2} \|\mathbf{w}\|^2$$

controls the margin size. Minimizing it produces the maximum-margin hyperplane.

The constraints ensure that all training samples are classified correctly.

How the Optimization is Solved

The above problem is a **convex quadratic optimization problem**. Such problems have a unique global optimum.

In practice the optimization is solved using methods such as

- Quadratic Programming
- Sequential Minimal Optimization (SMO)
- Gradient-based optimization methods

These algorithms compute the optimal parameters $\mathbf{w}$ and b that define the separating hyperplane.

An important result of the optimization is that only a small number of training points influence the final solution. These points are the **support vectors**. All other points do not affect the decision boundary.

4.2 Soft Margin SVM

Motivation

The hard-margin SVM assumes that the two classes can be perfectly separated by a linear boundary. In many real datasets this assumption is not realistic. Data may contain noise, measurement errors, or overlapping class distributions.

In such situations it may be impossible to find a hyperplane that correctly classifies every training point. Hard-margin SVM fails because its constraints require perfect separation.

To address this limitation, the **soft-margin SVM** allows some training points to violate the margin constraints. The classifier still tries to maximize the margin, but it also allows a limited number of classification errors.

Slack Variables

Soft-margin SVM introduces additional variables called **slack variables**. These variables measure how much a data point violates the margin constraint.

For each training example a slack variable ξ_i is introduced such that

$$\xi_i \geq 0.$$

The margin constraint becomes

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i.$$

The value of ξ_i indicates the position of the point relative to the margin.

- $\xi_i = 0$: the point is correctly classified and lies outside the margin
- $0 < \xi_i < 1$: the point lies inside the margin but remains correctly classified
- $\xi_i \geq 1$: the point is misclassified

Thus slack variables allow the classifier to tolerate margin violations.

Soft-Margin Optimization

The goal of SVM is still to find a hyperplane that maximizes the margin. At the same time the classifier should minimize the number of margin violations.

These two goals are combined into the following optimization problem:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

subject to

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, m.$$

The objective function contains two terms.

- The term $\frac{1}{2} \|\mathbf{w}\|^2$ maximizes the margin.
- The term $\sum \xi_i$ penalizes margin violations.

The optimization therefore balances margin size and classification errors.

Role of the Parameter C

The parameter $C > 0$ controls the relative importance of the two objectives.

- **Large C**

A large value of C strongly penalizes margin violations. The classifier tries to classify all training points correctly. This often produces a narrower margin and may lead to overfitting.

- **Small C**

A small value of C allows more violations of the margin. The classifier focuses more on maximizing the margin. This usually produces a wider margin and may improve generalization.

The value of C is usually selected using cross-validation.

Interpretation Using Loss Functions

Soft-margin SVM can also be interpreted using the **hinge loss**. The hinge loss for a training example is

$$L_i = \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

This loss is zero for correctly classified points outside the margin. It increases when a point moves inside the margin or becomes misclassified.

Using this formulation, the SVM optimization problem can also be written as

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

This representation is often used in modern machine learning implementations.

Geometric Illustration of SVM (See Figure 2)

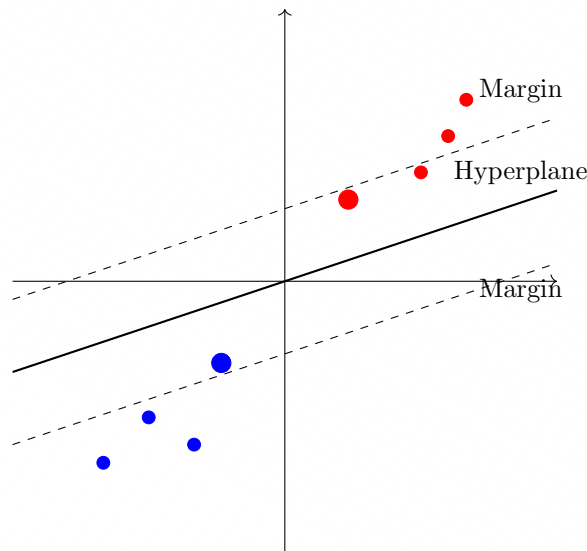


Figure 2: Geometric illustration of SVM showing the separating hyperplane (solid line), margin boundaries (dashed lines), and support vectors (highlighted points).

The solid line (in Figure 2) represents the separating hyperplane. The dashed lines represent the margin boundaries. The highlighted points closest to the margins are the support vectors.

Numerical Examples for SVM Margin

Problem 1: Computing the Margin

Problem. Consider the separating hyperplane

$$2x_1 + x_2 - 4 = 0.$$

Compute the margin of the SVM classifier.

Solution.

The hyperplane can be written in the standard form

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

Therefore

$$\mathbf{w} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad b = -4.$$

Step 1: Compute the norm of the weight vector

$$\|\mathbf{w}\| = \sqrt{2^2 + 1^2}$$

$$\|\mathbf{w}\| = \sqrt{4 + 1}$$

$$\|\mathbf{w}\| = \sqrt{5}$$

Step 2: Apply the SVM margin formula

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}$$

Step 3: Substitute the value

$$\text{margin} = \frac{2}{\sqrt{5}}$$

Final Answer

$$\text{margin} = \frac{2}{\sqrt{5}} \approx 0.894$$

Interpretation. The margin width is approximately 0.894. The distance from the hyperplane to each margin boundary is

$$\frac{1}{\|\mathbf{w}\|} = \frac{1}{\sqrt{5}} \approx 0.447.$$

This is the distance of the support vectors from the hyperplane.

Problem 2: Determining the Predicted Class

Problem. Suppose the SVM decision function is

$$f(\mathbf{x}) = 2x_1 + x_2 - 4.$$

Determine the predicted class of the point

$$\mathbf{x} = (3, 2).$$

Solution.

Step 1: Substitute the values

$$f(\mathbf{x}) = 2(3) + 2 - 4$$

Step 2: Compute the value

$$f(\mathbf{x}) = 6 + 2 - 4$$

$$f(\mathbf{x}) = 4$$

Step 3: Apply the classification rule

$$\hat{y} = \begin{cases} +1 & \text{if } f(\mathbf{x}) > 0 \\ -1 & \text{if } f(\mathbf{x}) < 0 \end{cases}$$

Since

$$f(\mathbf{x}) = 4 > 0$$

the predicted class is

$$\hat{y} = +1.$$

Interpretation. Because $f(\mathbf{x}) > 1$, the point lies outside the margin and is correctly classified.

Problem 3: Distance of a Point from the Hyperplane

Problem. Find the distance of the point

$$(1, 1)$$

from the hyperplane

$$2x_1 + x_2 - 4 = 0.$$

Solution.

The distance of a point from a hyperplane is

$$d = \frac{|w_1x_1 + w_2x_2 + b|}{\|\mathbf{w}\|}$$

where

$$\mathbf{w} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad b = -4.$$

Step 1: Substitute the coordinates

$$d = \frac{|2(1) + 1(1) - 4|}{\sqrt{2^2 + 1^2}}$$

Step 2: Simplify the numerator

$$d = \frac{|2 + 1 - 4|}{\sqrt{5}}$$

$$d = \frac{|-1|}{\sqrt{5}}$$

$$d = \frac{1}{\sqrt{5}}$$

Final Answer

$$d = \frac{1}{\sqrt{5}} \approx 0.447$$

Interpretation. The distance equals $\frac{1}{\|\mathbf{w}\|}$. Therefore the point lies exactly on the margin boundary. Points that lie on the margin boundaries are support vectors.

Problem 4: Checking the Margin Constraint

Problem. Consider the hyperplane

$$2x_1 + x_2 - 4 = 0.$$

Check whether the point

$$(3, 2)$$

with class label

$$y = +1$$

satisfies the SVM margin constraint.

Solution.

The margin constraint is

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$$

Step 1: Compute $\mathbf{w}^\top \mathbf{x} + b$

$$2(3) + 2 - 4$$

$$= 6 + 2 - 4$$

$$= 4$$

Step 2: Multiply by the class label

$$y(\mathbf{w}^\top \mathbf{x} + b)$$

$$= (+1)(4)$$

$$= 4$$

Step 3: Check the condition

$$4 \geq 1$$

Therefore the margin constraint is satisfied. The point is correctly classified and lies outside the margin.

Problem 5: Computing the Hyperplane from Two Support Vectors

Problem.

Consider two support vectors

$$\mathbf{x}_1 = (1, 1), \quad y_1 = +1$$

$$\mathbf{x}_2 = (3, 3), \quad y_2 = -1.$$

Assume that these points lie exactly on the margin boundaries of a linear SVM. Determine the equation of the separating hyperplane.

Solution.

The SVM decision function is

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b.$$

For support vectors

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1.$$

Step 1: Use the first support vector

$$(+1)(w_1(1) + w_2(1) + b) = 1$$

$$w_1 + w_2 + b = 1$$

Step 2: Use the second support vector

$$(-1)(w_1(3) + w_2(3) + b) = 1$$

$$3w_1 + 3w_2 + b = -1$$

Step 3: Subtract the equations

$$(3w_1 + 3w_2 + b) - (w_1 + w_2 + b) = -1 - 1$$

$$2w_1 + 2w_2 = -2$$

$$w_1 + w_2 = -1$$

Step 4: Compute the bias

$$(w_1 + w_2) + b = 1$$

$$-1 + b = 1$$

$$b = 2$$

Step 5: Choose a valid weight vector

One valid solution is

$$w_1 = -\frac{1}{2}, \quad w_2 = -\frac{1}{2}.$$

Step 6: Write the hyperplane

$$-\frac{1}{2}x_1 - \frac{1}{2}x_2 + 2 = 0$$

Multiply both sides by -2 :

$$x_1 + x_2 - 4 = 0.$$

Final Answer

The separating hyperplane is

$$x_1 + x_2 - 4 = 0.$$

Interpretation. The weight vector is $\mathbf{w} = [1, 1]^\top$ and the bias is $b = -4$. The margin width is

$$\frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{2}} = \sqrt{2}.$$

Each support vector lies at distance

$$\frac{1}{\|\mathbf{w}\|} = \frac{1}{\sqrt{2}}$$

from the separating hyperplane.

Problem 6: Identifying Support Vectors and Computing the SVM Boundary

Problem.

Consider the following four training points in a two-dimensional feature space.

$$(1, 1), \quad (2, 2) \quad \text{belong to class } +1$$

$$(4, 2), \quad (5, 3) \quad \text{belong to class } -1$$

Assume that the dataset is linearly separable.

1. Identify the support vectors.
2. Determine the equation of the separating hyperplane.

Solution.

Step 1: Identify the closest points between classes

The closest points between the two classes are

$$(2, 2) \quad \text{from class } +1$$

and

$$(4, 2) \quad \text{from class } -1.$$

These points determine the margin. Therefore they are the support vectors.

$$\mathbf{x}_1 = (2, 2), \quad y_1 = +1$$

$$\mathbf{x}_2 = (4, 2), \quad y_2 = -1$$

Step 2: Write the SVM margin conditions

For support vectors,

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1.$$

Step 3: Apply the condition to the first support vector

$$(+1)(2w_1 + 2w_2 + b) = 1$$

$$2w_1 + 2w_2 + b = 1$$

Step 4: Apply the condition to the second support vector

$$(-1)(4w_1 + 2w_2 + b) = 1$$

$$4w_1 + 2w_2 + b = -1$$

Step 5: Solve for w_1

Subtract the first equation from the second:

$$(4w_1 + 2w_2 + b) - (2w_1 + 2w_2 + b) = -1 - 1$$

$$2w_1 = -2$$

$$w_1 = -1$$

Step 6: Express b in terms of w_2

Substitute $w_1 = -1$ into the first equation:

$$2(-1) + 2w_2 + b = 1$$

$$-2 + 2w_2 + b = 1$$

$$2w_2 + b = 3$$

$$b = 3 - 2w_2$$

Step 7: Determine the maximum margin solution

The margin width is

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}$$

where

$$\|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2}.$$

Since $w_1 = -1$,

$$\|\mathbf{w}\| = \sqrt{1 + w_2^2}.$$

The margin becomes

$$\text{margin} = \frac{2}{\sqrt{1 + w_2^2}}.$$

The margin is maximized when $\sqrt{1 + w_2^2}$ is minimized. This occurs when

$$w_2 = 0.$$

Substitute $w_2 = 0$ into the expression for b :

$$b = 3 - 2(0) = 3.$$

Step 8: Write the hyperplane equation

$$\mathbf{w}^\top \mathbf{x} + b = -x_1 + 3 = 0$$

or

$$x_1 = 3.$$

Step 9: Verify the remaining points

For $(1, 1)$ with $y = +1$:

$$f(1, 1) = -1(1) + 3 = 2$$

$$yf(\mathbf{x}) = 2 > 1$$

For $(5, 3)$ with $y = -1$:

$$f(5, 3) = -5 + 3 = -2$$

$$yf(\mathbf{x}) = 2 > 1$$

Both points lie outside the margin and are not support vectors.

Final Answer

The separating hyperplane is

$$x_1 - 3 = 0.$$

The support vectors are

(2, 2) and (4, 2).

Interpretation.

The hyperplane lies exactly halfway between the support vectors at $x_1 = 3$.
The margin width is

$$\frac{2}{\|\mathbf{w}\|} = \frac{2}{1} = 2.$$

The distance from the hyperplane to each support vector is

$$\frac{1}{\|\mathbf{w}\|} = 1.$$

The other two points do not affect the hyperplane because they lie outside the margin.

5 Instance-Based Learning: k -Nearest Neighbors (k-NN)

Introduction

Instance-based learning is a type of learning in which the model does not explicitly learn a general rule during training. Instead, it stores the training data and makes predictions by comparing new samples with stored instances. This approach is also called **lazy learning** because generalization occurs only when a prediction is required.

The **k -Nearest Neighbors (k-NN)** method is a simple and widely used instance-based learning algorithm.

Basic Idea of k-NN

The main idea of k-NN is as follows:

- Given a new input point, find the k closest points in the training dataset.
- Use the labels of these k neighbors to predict the class (for classification) or average their values (for regression).

The closeness is measured using a distance metric.

Distance Measures

The most common distance measure is the Euclidean distance:

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{i=1}^n (x_i - z_i)^2}.$$

Other commonly used distance measures include:

- Manhattan distance:

$$d(\mathbf{x}, \mathbf{z}) = \sum_{i=1}^n |x_i - z_i|$$

- Minkowski distance:

$$d(\mathbf{x}, \mathbf{z}) = \left(\sum_{i=1}^n |x_i - z_i|^p \right)^{1/p}$$

- Cosine distance:

$$d(\mathbf{x}, \mathbf{z}) = 1 - \frac{\mathbf{x}^\top \mathbf{z}}{\|\mathbf{x}\| \|\mathbf{z}\|}$$

5.1 Algorithm for k-NN Classification

1. Choose the value of k .
2. Compute the distance between the test point and all training points.
3. Select the k nearest neighbors.
4. Count the number of samples in each class among the neighbors.
5. Assign the class with the highest count.

For regression, the prediction is the average of the target values of the k neighbors.

Weighted k-NN

In the basic k-NN method, all k neighbors contribute equally to the prediction. However, this may not be ideal because closer neighbors are usually more relevant than distant ones.

Weighted k-NN assigns higher importance to nearer neighbors and lower importance to farther neighbors.

Let $\mathbf{x}$ be the test point. Let $\mathcal{N}_k(\mathbf{x})$ denote the set of indices of the k nearest neighbors of $\mathbf{x}$. Each neighbor $\mathbf{x}_i$ is assigned a weight w_i based on its distance from $\mathbf{x}$. A common choice is

$$w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)},$$

where $d(\mathbf{x}, \mathbf{x}_i)$ is the distance between the test point and the i -th neighbor.

Weighted classification rule:

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x}), y_i = c} w_i$$

where

- $\hat{y}$ is the predicted class
- c represents a possible class label
- $\mathcal{N}_k(\mathbf{x})$ is the set of k nearest neighbors
- y_i is the class label of the i -th neighbor
- w_i is the weight assigned to neighbor i

This rule computes the total weight for each class and assigns the class with the largest total weight.

Weighted regression:

For regression, the predicted value is a weighted average:

$$\hat{y} = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i y_i}{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i}.$$

Remarks:

- Closer neighbors have larger weights and influence the prediction more strongly.
- To avoid division by zero when distance is very small, a small constant ϵ can be added:

$$w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i) + \epsilon}.$$

Example

Let $k = 3$. Suppose the nearest neighbors have labels

$$\{+1, +1, -1\}.$$

The majority class is $+1$, so the predicted class is $+1$.

For weighted k-NN with distances $\{0.5, 0.6, 0.2\}$:

The total weight for class $+1$ is

$$\frac{1}{0.5} + \frac{1}{0.6} = 2 + 1.67 = 3.67.$$

The total weight for class -1 is

$$\frac{1}{0.2} = 5.$$

Thus the predicted class is -1 .

Choice of k

- Small k : sensitive to noise and may overfit
- Large k : smoother decision boundary but may underfit
- The value of k is usually selected using cross-validation
- Odd values of k help avoid ties in binary classification

Feature Scaling

k-NN is sensitive to the scale of features. Features with larger values dominate distance computation. Standardization or normalization is required before applying k-NN.

Advantages of k-NN

- Simple to understand and implement
- No training phase
- Naturally handles multi-class classification
- Adapts to local data structure

Limitations of k-NN

- High computational cost during prediction ($O(mn)$, where m is the number of training samples and n is the number of features)
- Requires storing all training data
- Sensitive to feature scaling and irrelevant features
- Performance decreases in high dimensions

Curse of Dimensionality

In high-dimensional spaces, distances become less meaningful. Many points appear at similar distances from each other, which reduces the effectiveness of k-NN. Dimensionality reduction techniques can help address this issue.

Numerical Problem 1: k-NN Classification (Euclidean Distance)

Problem.

Consider the following dataset of five training samples with two features x_1 and x_2 , and a binary class label (Pass or Fail):

x_1	x_2	Class
14	28	Pass
22	35	Fail
27	24	Pass
31	38	Fail
20	30	Pass

Classify the test point $\mathbf{x} = (25, 29)$ using the k -Nearest Neighbors algorithm with $k = 3$ and Euclidean distance.

Solution.

Step 1: Compute Euclidean distances

The Euclidean distance between two points $\mathbf{x} = (x_1, x_2)$ and $\mathbf{z} = (z_1, z_2)$ is given by

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}.$$

For each training point, compute the distance to the test point $(25, 29)$.

Point 1: (14, 28), Class: Pass

$$d_1 = \sqrt{(25 - 14)^2 + (29 - 28)^2} = \sqrt{11^2 + 1^2} = \sqrt{121 + 1} = \sqrt{122}$$

$$d_1 \approx 11.05$$

Point 2: (22, 35), Class: Fail

$$d_2 = \sqrt{(25 - 22)^2 + (29 - 35)^2} = \sqrt{3^2 + (-6)^2} = \sqrt{9 + 36} = \sqrt{45}$$

$$d_2 \approx 6.71$$

Point 3: (27, 24), Class: Pass

$$d_3 = \sqrt{(25 - 27)^2 + (29 - 24)^2} = \sqrt{(-2)^2 + 5^2} = \sqrt{4 + 25} = \sqrt{29}$$

$$d_3 \approx 5.39$$

Point 4: (31, 38), Class: Fail

$$d_4 = \sqrt{(25 - 31)^2 + (29 - 38)^2} = \sqrt{(-6)^2 + (-9)^2} = \sqrt{36 + 81} = \sqrt{117}$$

$$d_4 \approx 10.82$$

Point 5: (20, 30), Class: Pass

$$d_5 = \sqrt{(25 - 20)^2 + (29 - 30)^2} = \sqrt{5^2 + (-1)^2} = \sqrt{25 + 1} = \sqrt{26}$$

$$d_5 \approx 5.10$$

Step 2: Summarize distances

Point	Coordinates	Class	Distance
1	(14, 28)	Pass	$\sqrt{122} \approx 11.05$
2	(22, 35)	Fail	$\sqrt{45} \approx 6.71$
3	(27, 24)	Pass	$\sqrt{29} \approx 5.39$
4	(31, 38)	Fail	$\sqrt{117} \approx 10.82$
5	(20, 30)	Pass	$\sqrt{26} \approx 5.10$

Step 3: Select the $k = 3$ nearest neighbors

Sort the points by distance (smallest to largest):

Rank	Distance	Class
1	5.10	Pass
2	5.39	Pass
3	6.71	Fail
4	10.82	Fail
5	11.05	Pass

The three nearest neighbors are:

- Point 5: (20, 30) with class Pass
- Point 3: (27, 24) with class Pass
- Point 2: (22, 35) with class Fail

Step 4: Majority voting

Count the occurrences of each class among the $k = 3$ nearest neighbors:

Pass count = 2 (Point 5 and Point 3)

Fail count = 1 (Point 2)

Since the class Pass has the majority (2 votes out of 3), the test point is classified as Pass.

Step 5: Final answer

$$\hat{y} = \text{Pass}$$

Interpretation: The test point (25, 29) is closest to training points that are predominantly from the Pass class. The three nearest neighbors include two Pass samples and one Fail sample, leading to a majority vote in favor of Pass.

Numerical Problem 2: k-NN Classification with Euclidean Distance**Problem.**

Consider the following training data with two features x_1 and x_2 , and binary class labels $y \in \{-1, +1\}$:

x_1	x_2	y
2.5	3.1	+1
1.2	0.7	-1
3.6	2.8	+1
4.0	4.5	-1
2.8	2.0	+1

Classify the test point $\mathbf{x} = (2.9, 2.7)$ using the k -Nearest Neighbors algorithm with $k = 3$ and Euclidean distance.

Solution.**Step 1: Compute Euclidean distances**

The Euclidean distance between two points $\mathbf{x} = (x_1, x_2)$ and $\mathbf{z} = (z_1, z_2)$ is

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}.$$

For each training point, compute the distance to the test point (2.9, 2.7).

Point 1: (2.5, 3.1), Class: +1

$$d_1 = \sqrt{(2.9 - 2.5)^2 + (2.7 - 3.1)^2} = \sqrt{0.4^2 + (-0.4)^2} = \sqrt{0.16 + 0.16} = \sqrt{0.32} \approx 0.566$$

Point 2: (1.2, 0.7), Class: -1

$$d_2 = \sqrt{(2.9 - 1.2)^2 + (2.7 - 0.7)^2} = \sqrt{1.7^2 + 2.0^2} = \sqrt{2.89 + 4.00} = \sqrt{6.89} \approx 2.626$$

Point 3: (3.6, 2.8), Class: +1

$$d_3 = \sqrt{(2.9 - 3.6)^2 + (2.7 - 2.8)^2} = \sqrt{(-0.7)^2 + (-0.1)^2} = \sqrt{0.49 + 0.01} = \sqrt{0.50} \approx 0.707$$

Point 4: (4.0, 4.5), Class: -1

$$d_4 = \sqrt{(2.9 - 4.0)^2 + (2.7 - 4.5)^2} = \sqrt{(-1.1)^2 + (-1.8)^2} = \sqrt{1.21 + 3.24} = \sqrt{4.45} \approx 2.109$$

Point 5: (2.8, 2.0), Class: +1

$$d_5 = \sqrt{(2.9 - 2.8)^2 + (2.7 - 2.0)^2} = \sqrt{0.1^2 + 0.7^2} = \sqrt{0.01 + 0.49} = \sqrt{0.50} \approx 0.707$$

Step 2: Summarize distances

Point	Coordinates	Distance
1	(2.5, 3.1)	+1
2	(1.2, 0.7)	-1
3	(3.6, 2.8)	+1
4	(4.0, 4.5)	-1
5	(2.8, 2.0)	+1

Step 3: Select the $k = 3$ nearest neighbors

Sort the points by distance (smallest to largest):

Rank	Distance	Class
1	0.566	+1
2	0.707	+1
3	0.707	+1
4	2.109	-1
5	2.626	-1

The three nearest neighbors are:

- Point 1: (2.5, 3.1) with class +1
- Point 3: (3.6, 2.8) with class +1
- Point 5: (2.8, 2.0) with class +1

Step 4: Majority voting

Count the occurrences of each class among the $k = 3$ nearest neighbors:

Class +1 count = 3

Class -1 count = 0

The majority class is +1.

$$\hat{y} = +1$$

Interpretation: All three nearest neighbors belong to class +1. Therefore the test point is classified as +1. The closest neighbor is Point 1 at distance 0.566, followed by Points 3 and 5 at distance 0.707.

Numerical Problem 3: k-NN Classification (Euclidean Distance)

Problem.

Consider the following dataset of five training samples with two features x_1 and x_2 , and a binary class label (Urban or Rural):

x_1	x_2	Class
35	45	Urban
42	38	Rural
28	50	Urban
50	30	Rural
40	48	Urban

Classify the test point $\mathbf{x} = (38, 42)$ using the k -Nearest Neighbors algorithm with $k = 3$ and Euclidean distance.

Solution.

Step 1: Compute Euclidean distances

The Euclidean distance between two points $\mathbf{x} = (x_1, x_2)$ and $\mathbf{z} = (z_1, z_2)$ is

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}.$$

For each training point, compute the distance to the test point $(38, 42)$.

Point 1: $(35, 45)$, Class: Urban

$$d_1 = \sqrt{(38 - 35)^2 + (42 - 45)^2} = \sqrt{3^2 + (-3)^2} = \sqrt{9 + 9} = \sqrt{18} \approx 4.24$$

Point 2: $(42, 38)$, Class: Rural

$$d_2 = \sqrt{(38 - 42)^2 + (42 - 38)^2} = \sqrt{(-4)^2 + 4^2} = \sqrt{16 + 16} = \sqrt{32} \approx 5.66$$

Point 3: $(28, 50)$, Class: Urban

$$d_3 = \sqrt{(38 - 28)^2 + (42 - 50)^2} = \sqrt{10^2 + (-8)^2} = \sqrt{100 + 64} = \sqrt{164} \approx 12.81$$

Point 4: $(50, 30)$, Class: Rural

$$d_4 = \sqrt{(38 - 50)^2 + (42 - 30)^2} = \sqrt{(-12)^2 + 12^2} = \sqrt{144 + 144} = \sqrt{288} \approx 16.97$$

Point 5: $(40, 48)$, Class: Urban

$$d_5 = \sqrt{(38 - 40)^2 + (42 - 48)^2} = \sqrt{(-2)^2 + (-6)^2} = \sqrt{4 + 36} = \sqrt{40} \approx 6.32$$

Step 2: Summarize distances

Point	Coordinates	Distance
1	(35, 45) Urban	$\sqrt{18} \approx 4.24$
2	(42, 38) Rural	$\sqrt{32} \approx 5.66$
3	(28, 50) Urban	$\sqrt{164} \approx 12.81$
4	(50, 30) Rural	$\sqrt{288} \approx 16.97$
5	(40, 48) Urban	$\sqrt{40} \approx 6.32$

Step 3: Select the $k = 3$ nearest neighbors

Sort the points by distance (smallest to largest):

Rank	Distance	Class
1	4.24	Urban
2	5.66	Rural
3	6.32	Urban
4	12.81	Urban
5	16.97	Rural

The three nearest neighbors are:

- Point 1: (35, 45) with class Urban
- Point 2: (42, 38) with class Rural
- Point 5: (40, 48) with class Urban

Step 4: Majority voting

Count the occurrences of each class among the $k = 3$ nearest neighbors:

Urban count = 2 (Point 1 and Point 5)

Rural count = 1 (Point 2)

The majority class is Urban.

$$\hat{y} = \text{Urban}$$

Interpretation: The test point (38, 42) is closest to the Urban point (35, 45) at distance 4.24. Among the three nearest neighbors, two are Urban and one is Rural. Therefore the test point is classified as Urban.

Numerical Problem 1 (Optional): Weighted k-NN Classification

Problem.

Consider the following dataset of five training samples with two features x_1 and x_2 , and a binary class label (Approved or Rejected):

x_1	x_2	Class
18	26	Approved
24	32	Rejected
29	27	Approved
33	36	Rejected
21	30	Approved

Classify the test point $\mathbf{x} = (26, 31)$ using weighted k-NN with $k = 3$ and weights $w_i = \frac{1}{d_i}$, where d_i is the Euclidean distance to the test point.

Solution.

Step 1: Compute Euclidean distances

The Euclidean distance between two points $\mathbf{x} = (x_1, x_2)$ and $\mathbf{z} = (z_1, z_2)$ is

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}.$$

For each training point, compute the distance to the test point (26, 31).

Point 1: (18, 26), Class: Approved

$$d_1 = \sqrt{(26 - 18)^2 + (31 - 26)^2} = \sqrt{8^2 + 5^2} = \sqrt{64 + 25} = \sqrt{89} \approx 9.43$$

Point 2: (24, 32), Class: Rejected

$$d_2 = \sqrt{(26 - 24)^2 + (31 - 32)^2} = \sqrt{2^2 + (-1)^2} = \sqrt{4 + 1} = \sqrt{5} \approx 2.24$$

Point 3: (29, 27), Class: Approved

$$d_3 = \sqrt{(26 - 29)^2 + (31 - 27)^2} = \sqrt{(-3)^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Point 4: (33, 36), Class: Rejected

$$d_4 = \sqrt{(26 - 33)^2 + (31 - 36)^2} = \sqrt{(-7)^2 + (-5)^2} = \sqrt{49 + 25} = \sqrt{74} \approx 8.60$$

Point 5: (21, 30), Class: Approved

$$d_5 = \sqrt{(26 - 21)^2 + (31 - 30)^2} = \sqrt{5^2 + 1^2} = \sqrt{25 + 1} = \sqrt{26} \approx 5.10$$

Step 2: Summarize distances

Point	Coordinates		Distance
1	(18, 26)	Approved	$\sqrt{89} \approx 9.43$
2	(24, 32)	Rejected	$\sqrt{5} \approx 2.24$
3	(29, 27)	Approved	5.00
4	(33, 36)	Rejected	$\sqrt{74} \approx 8.60$
5	(21, 30)	Approved	$\sqrt{26} \approx 5.10$

Step 3: Select the $k = 3$ nearest neighbors

Sort the points by distance (smallest to largest):

Rank	Distance	Class
1	2.24	Rejected
2	5.00	Approved
3	5.10	Approved
4	8.60	Rejected
5	9.43	Approved

The three nearest neighbors are:

- Point 2: (24, 32) with class Rejected (distance 2.24)
- Point 3: (29, 27) with class Approved (distance 5.00)
- Point 5: (21, 30) with class Approved (distance 5.10)

Step 4: Compute weights

Weights are computed as $w_i = \frac{1}{d_i}$:

$$w_2 = \frac{1}{2.24} \approx 0.446$$

$$w_3 = \frac{1}{5.00} = 0.200$$

$$w_5 = \frac{1}{5.10} \approx 0.196$$

Step 5: Compute weighted votes

Sum the weights for each class:

- **Approved class** (Points 3 and 5):

$$\text{Weighted vote}_{\text{Approved}} = 0.200 + 0.196 = 0.396$$

- **Rejected class** (Point 2):

$$\text{Weighted vote}_{\text{Rejected}} = 0.446$$

Step 6: Determine predicted class

Compare the weighted votes:

$$0.446 > 0.396$$

The Rejected class has a higher total weight.

$$\hat{y} = \text{Rejected}$$

Interpretation

Although two of the three nearest neighbors belong to the Approved class, the closest neighbor (Rejected) receives the highest weight due to its smaller distance (2.24 versus 5.00 and 5.10). This gives the Rejected class a larger total weighted vote, leading to a Rejected prediction.

Comparison with Standard k-NN

For standard k-NN with $k = 3$ (equal weights), the three nearest neighbors are Rejected, Approved, Approved. Majority voting gives 2 votes for Approved and 1 vote for Rejected, so the prediction would be Approved.

Weighted k-NN reverses this decision because the Rejected point is significantly closer than the two Approved points, giving it more influence. This demonstrates how weighted k-NN can be more sensitive to local structure than standard k-NN.

Numerical Problem 2 (Optional): Weighted k-NN Classification

Problem.

Consider the following nearest neighbors of a test point, along with their distances and class labels:

Distance	Class
0.82	+1
1.15	-1
0.64	+1
0.58	-1

Use weighted k-NN with $k = 4$ and weights inversely proportional to distance ($w_i = 1/d_i$). Determine the predicted class.

Solution.

Step 1: Compute weights

For each neighbor, compute the weight $w_i = \frac{1}{d_i}$:

$$w_1 = \frac{1}{0.82} \approx 1.2195$$

$$w_2 = \frac{1}{1.15} \approx 0.8696$$

$$w_3 = \frac{1}{0.64} \approx 1.5625$$

$$w_4 = \frac{1}{0.58} \approx 1.7241$$

For clarity, round to two decimal places:

$$w_1 \approx 1.22, \quad w_2 \approx 0.87, \quad w_3 \approx 1.56, \quad w_4 \approx 1.72$$

Step 2: Organize data with weights

Neighbor	Distance	Weight	Class
1	0.82	1.22	+1
2	1.15	0.87	-1
3	0.64	1.56	+1
4	0.58	1.72	-1

Step 3: Sum weights by class

Class +1 (Neighbors 1 and 3):

$$\text{Weighted vote}_{+1} = 1.22 + 1.56 = 2.78$$

Class -1 (Neighbors 2 and 4):

$$\text{Weighted vote}_{-1} = 0.87 + 1.72 = 2.59$$

Step 4: Determine predicted class

Compare the weighted votes:

$$2.78 > 2.59$$

The class +1 has a higher total weight.

$$\hat{y} = +1$$

Interpretation: There are two neighbors from each class (+1 and -1). However, the weighted votes favor +1 because the +1 neighbors have smaller distances (0.64 and 0.82) compared to the -1 neighbors (0.58 and 1.15). The closest neighbor (distance 0.58) belongs to class -1 and receives weight 1.72, but this is outweighed by the combined weights of the two +1 neighbors ($1.22 + 1.56 = 2.78$).

Comparison with Standard k-NN

For standard k-NN with $k = 4$ (equal weights), each neighbor gets one vote:

- Class +1: 2 votes
- Class -1: 2 votes

This results in a tie. Weighted k-NN resolves the tie by considering the distances, leading to a +1 prediction.

Method	Class +1	Class -1
Standard k-NN	2 votes	2 votes (tie)
Weighted k-NN	2.78	2.59

Numerical Problem 1: k-NN for Regression

Problem.

Consider the following dataset with two features x_1 and x_2 and a continuous target variable y :

x_1	x_2	y
15	20	55
22	18	60
30	25	72
28	22	68
18	24	58

Predict the target value at the test point $\mathbf{x} = (24, 21)$ using the k -Nearest Neighbors regression algorithm with $k = 3$ and Euclidean distance.

Solution.

Step 1: Compute Euclidean distances

The Euclidean distance between two points $\mathbf{x} = (x_1, x_2)$ and $\mathbf{z} = (z_1, z_2)$ is

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}.$$

For each training point, compute the distance to the test point $(24, 21)$.

Point 1: (15, 20), $y = 55$

$$d_1 = \sqrt{(24 - 15)^2 + (21 - 20)^2} = \sqrt{9^2 + 1^2} = \sqrt{81 + 1} = \sqrt{82} \approx 9.06$$

Point 2: (22, 18), $y = 60$

$$d_2 = \sqrt{(24 - 22)^2 + (21 - 18)^2} = \sqrt{2^2 + 3^2} = \sqrt{4 + 9} = \sqrt{13} \approx 3.61$$

Point 3: (30, 25), $y = 72$

$$d_3 = \sqrt{(24 - 30)^2 + (21 - 25)^2} = \sqrt{(-6)^2 + (-4)^2} = \sqrt{36 + 16} = \sqrt{52} \approx 7.21$$

Point 4: (28, 22), $y = 68$

$$d_4 = \sqrt{(24 - 28)^2 + (21 - 22)^2} = \sqrt{(-4)^2 + (-1)^2} = \sqrt{16 + 1} = \sqrt{17} \approx 4.12$$

Point 5: (18, 24), $y = 58$

$$d_5 = \sqrt{(24 - 18)^2 + (21 - 24)^2} = \sqrt{6^2 + (-3)^2} = \sqrt{36 + 9} = \sqrt{45} \approx 6.71$$

Step 2: Summarize distances

Point	Coordinates	Distance
1	(15, 20)	55
2	(22, 18)	60
3	(30, 25)	72
4	(28, 22)	68
5	(18, 24)	58

Step 3: Select the $k = 3$ nearest neighbors

Sort the points by distance (smallest to largest):

Rank	Distance	y Value
1	3.61	60
2	4.12	68
3	6.71	58
4	7.21	72
5	9.06	55

The three nearest neighbors are:

- Point 2: (22, 18) with $y = 60$
- Point 4: (28, 22) with $y = 68$
- Point 5: (18, 24) with $y = 58$

Step 4: Compute the average for regression

For regression, the prediction is the average of the target values of the k nearest neighbors:

$$\hat{y} = \frac{60 + 68 + 58}{3} = \frac{186}{3} = 62$$

$$\boxed{\hat{y} = 62}$$

Interpretation: The test point (24, 21) is closest to Point 2 (distance 3.61), followed by Point 4 (4.12) and Point 5 (6.71). The average of their target values is 62, which is the predicted output.

Numerical Problem 3: k-NN Regression (Second Example)

Problem.

Consider the following dataset:

x_1	x_2	y
16	22	65
23	19	70
28	26	82
31	24	78
19	27	68

Predict the target value at the test point $\mathbf{x} = (26, 23)$ using $k = 3$.

Solution.

Step 1: Compute Euclidean distances

For each training point, compute the distance to the test point $(26, 23)$.

Point 1: $(16, 22)$, $y = 65$

$$d_1 = \sqrt{(26 - 16)^2 + (23 - 22)^2} = \sqrt{10^2 + 1^2} = \sqrt{100 + 1} = \sqrt{101} \approx 10.05$$

Point 2: $(23, 19)$, $y = 70$

$$d_2 = \sqrt{(26 - 23)^2 + (23 - 19)^2} = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Point 3: $(28, 26)$, $y = 82$

$$d_3 = \sqrt{(26 - 28)^2 + (23 - 26)^2} = \sqrt{(-2)^2 + (-3)^2} = \sqrt{4 + 9} = \sqrt{13} \approx 3.61$$

Point 4: $(31, 24)$, $y = 78$

$$d_4 = \sqrt{(26 - 31)^2 + (23 - 24)^2} = \sqrt{(-5)^2 + (-1)^2} = \sqrt{25 + 1} = \sqrt{26} \approx 5.10$$

Point 5: $(19, 27)$, $y = 68$

$$d_5 = \sqrt{(26 - 19)^2 + (23 - 27)^2} = \sqrt{7^2 + (-4)^2} = \sqrt{49 + 16} = \sqrt{65} \approx 8.06$$

Step 2: Summarize distances

Point	Coordinates	Distance
1	(16, 22)	65
2	(23, 19)	70
3	(28, 26)	82
4	(31, 24)	78
5	(19, 27)	68

Step 3: Select the $k = 3$ nearest neighbors

Sort the points by distance:

Rank	Distance	y Value
1	3.61	82
2	5.00	70
3	5.10	78
4	8.06	68
5	10.05	65

The three nearest neighbors are:

- Point 3: $(28, 26)$ with $y = 82$

- Point 2: (23, 19) with $y = 70$
- Point 4: (31, 24) with $y = 78$

Step 4: Compute the average for regression

$$\hat{y} = \frac{82 + 70 + 78}{3} = \frac{230}{3} \approx 76.67$$

$$\boxed{\hat{y} \approx 76.67}$$

Interpretation: The test point (26, 23) is closest to Point 3 (distance 3.61), followed by Point 2 (5.00) and Point 4 (5.10). The average of their target values is approximately 76.67, which is the predicted output.

6 Ensemble Methods

Introduction

Ensemble methods combine multiple models (often called **base learners** or **weak learners**) to improve overall performance. The core idea is that a group of models can produce better results than any single model, analogous to the "wisdom of the crowd."

6.1 Bias-Variance Trade-off

Ensemble methods work by managing the bias-variance trade-off:

- **Bagging** reduces **variance** by averaging predictions from models trained on different data subsets.
- **Boosting** reduces **bias** by sequentially focusing on difficult examples.

Bagging (Bootstrap Aggregating)

Bagging trains multiple models on different subsets of the data.

1. Generate bootstrap samples from the dataset.
2. Train a model on each sample.
3. Combine predictions using voting (classification) or averaging (regression).

Models are trained independently and in parallel.

Bias, Variance, and the Bias-Variance Trade-off

When a machine learning model makes predictions, errors come from two main sources:

- **Bias:** Error from oversimplifying the problem. High bias means the model misses important patterns (underfitting).
- **Variance:** Error from being too sensitive to training data. High variance means the model learns noise instead of patterns (overfitting).

The Trade-off

Simple models have high bias but low variance. Complex models have low bias but high variance. The goal is to find the sweet spot where total error is minimized.

Model	Bias	Variance
Simple (e.g., linear)	High	Low
Complex (e.g., deep tree)	Low	High
Optimal	Balanced	Balanced

Example: k-NN

- Small k : low bias, high variance (overfits)
- Large k : high bias, low variance (underfits)

We balance bias and variance to build models that generalize well to new data.

Boosting

Boosting trains models sequentially. Each new model focuses on correcting the mistakes made by previous models.

A **weak learner** is a simple model that performs slightly better than random guessing.

1. Assign equal weights to all samples.
2. Train a model.
3. Increase weights of misclassified samples.
4. Train the next model.
5. Repeat for several iterations.
6. Combine all models into a final prediction.

6.2 Random Forest

Random Forest is an ensemble of decision trees.

- Each tree is trained on a bootstrap sample.
- At each split, a random subset of features is used.
- Final prediction is obtained by voting or averaging.

Random feature selection reduces correlation between trees and improves performance.

Advantages of Random Forest

- Reduces overfitting
- Works well for high-dimensional data
- Provides feature importance
- Robust to noise and outliers

Comparison of Ensemble Methods

Method	Training	Main Effect	Example
Bagging	Parallel	Reduces variance	Random Forest
Boosting	Sequential	Reduces bias	AdaBoost, Gradient Boosting
Stacking	Parallel + Meta	Combines diverse models	Any combination
Voting	Parallel	Simple combination	Ensemble of classifiers

Practical Considerations

- Diversity of models improves ensemble performance
- Ensemble methods increase computational cost
- Ensembles are less interpretable than single models